

Esercizio A1

Limite superiore Andando per tentativi, proviamo con $\Theta(n^2)$. Proviamo quindi a dimostrare che $T(n) = O(n^2)$.

- Ipotesi induttiva: $T(k) \leq ck^2$, per $k < n$
- Passo induttivo:

$$\begin{aligned} T(n) &= 2T(\lfloor n/3 \rfloor) + 3T(\lfloor n/2 \rfloor) + n^2 \\ &\leq 2c\lfloor n/3 \rfloor^2 + 3c\lfloor n/2 \rfloor^2 + n^2 \\ &\leq \frac{2}{9}cn^2 + \frac{3}{4}cn^2 + n^2 \\ &= \frac{35}{36}cn^2 + n^2 \\ &\leq cn^2 \end{aligned}$$

L'ultima disequazione è vera per $c \geq 36$.

- Casi base:

$$T(1) = 1 \leq c \cdot 1^2 \Rightarrow c \geq 1$$

$$T(2) = 1 \leq c \cdot 2^2 \Rightarrow c \geq 1/4$$

Abbiamo quindi dimostrato che $T(n) = O(n^2)$, con $m = 1$ e $c \geq 36$.

È facile dimostrare che $T(n) = \Omega(n^2)$ per via della componente non ricorsiva n^2 .

Abbiamo quindi dimostrato che $T(n) = \Theta(n^2)$.

Esercizio A2

Per risolvere questo esercizio, è possibile utilizzare una singola visita in profondità dell'albero, che però raccoglie due informazioni separate.

Per ogni nodo u , vogliamo misurare:

- il costo più alto per andare da u ad una qualunque delle sue foglie (*highest*);
- il costo massimo fra tutti i cammini semplici contenuti nel sottoalbero radicato in u che uniscono due foglie (*maxpath*).

Calcolare *highest* di un nodo u è semplice: basta prendere il massimo fra il valore *highest* dei figli sinistro e destro, e sommare il peso $u.weight$. Se u è **nil**, si restituisce 0.

Per calcolare *maxpath* di un albero radicato in u , si prende il massimo fra:

- Il *maxpath* del sottoalbero sinistro;
- Il *maxpath* del sottoalbero destro;
- Il costo del cammino massimale che passa attraverso il nodo u , calcolato come il costo più alto per andare da $u.left()$ ad una delle sue foglie, da $u.right()$ ad una delle sue foglie, più il peso del nodo u .

Ci sono alcuni casi particolari:

- Nel caso $u == \mathbf{nil}$, si restituisce $-\infty$ per entrambi i valori; si assume che in questo modo, se un nodo u non ha figlio destro oppure sinistro, la somma $highest_L + highest_R + T.weight$ darà comunque $-\infty$ e non verrà selezionata dalla funzione **max**;

- Nel caso u sia una foglia, si restituisce il peso del nodo sia come *highest* che come *maxpath*.

In ogni caso, il codice può essere reso più chiaro "sviscerando" ognuno dei quattro casi: **nil**, foglia, solo figlio sinistro, solo figlio destro, due figli.

```
int maxPath(TREE  $T$ )
```

```
    highest, maxpath = maxPathRec( $T$ )
    return maxpath
```

```
(int, int) maxPathRec(TREE  $T$ )
```

```
if  $T == \text{nil}$  then
    | return  $(-\infty, -\infty)$ ;
if  $T.\text{left}() == \text{nil}$  and  $T.\text{right}() == \text{nil}$  then
    | return  $(T.\text{weight}, T.\text{weight})$ ;
highest $L$ , maxpath $L$  = maxPath( $T.\text{left}()$ )
highest $R$ , maxpath $R$  = maxPath( $T.\text{right}()$ )
highest =  $\max(\text{highest}_L, \text{highest}_R) + T.\text{weight}$ 
maxpath =  $\max(\text{maxpath}_L, \text{maxpath}_R, \text{highest}_L + \text{highest}_R + T.\text{weight})$ 
return highest, maxpath
```

La complessità dell'algoritmo proposto è quella di una visita in profondità di un albero, ovvero $\Theta(n)$.

Esercizio A3

In modo simile al compito dell'Agosto 2018, il problema richiede di individuare le componenti connesse nel grafo dove le caselle adiacenti sono connesse da un arco. In questo caso, è sufficiente individuare la sola componente connessa che contiene la cella di input (r, c) .

Per risolvere il problema, viene fatta una visita in profondità a partire dalla cella (r, c) , progettata in modo tale che restituisca il numero di celle della linea costiera che sono state incontrate durante la visita.

Quando viene visitata una cella (r, c) valida che non è stata visitata in precedenza, la cella viene marcata come visitata; possono darsi due casi:

- Se è una cella di mare ($M[r][c] < 0$), si visitano tutti i nodi adiacenti a partire da essa (up, down, left, right) e si restituisce la somma delle celle di linea costiera che sono state incontrate
- Se è una cella di terra ($M[r][c] \geq 0$), si restituisce 1 perché è stata visita a partire da una cella di mare e quindi fa parte della linea costiera.

L'algoritmo ha costo $O(n^2)$, per via dell'inizializzazione di *visited* e per il fatto che ogni cella viene visitata al più una volta.

```
int coastLen(int[][]  $M$ , int  $n$ , int  $r$ , int  $c$ )
```

```
    boolean[][] visited = new boolean[ $0 \dots n - 1$ ][ $0 \dots n - 1$ ]
    for  $r = 0$  to  $n - 1$  do
        | for  $c = 0$  to  $n - 1$  do
            | | visited[ $r$ ][ $c$ ] = false
    return coastDFS( $M$ , visited,  $n$ ,  $r$ ,  $c$ )
```

```
int coastDFS(int[][] M, boolean[][] visited, int n, int r, int c)
```

```
  if r < 0 or r ≥ n or c < 0 or c ≥ n or visited[r][c] then  
    return 0
```

```
  visited[r][c] = true
```

```
  if M[r][c] < 0 then
```

```
    int tU = coastDFS(m, visited, r − 1, c)
```

```
    int tD = coastDFS(m, visited, r + 1, c)
```

```
    int tL = coastDFS(m, visited, r, c − 1)
```

```
    int tR = coastDFS(m, visited, r, c + 1)
```

```
    return tU + tD + tL + tR
```

```
  else
```

```
    return 1
```
